

Informe de Pruebas Unitarias

Proyecto: GamePeruleto | Integrantes: Alex Martínez Martins / Francisco José O'Bryon Olmeda | Módulo: Entornos de Desarrollo

Identificación del Objeto de Prueba:

- Clase: `Gestor` (Modelo)
- Método: `listar()`
- Técnica: Caja Blanca (Camino Básico)

1. Descripción del Método

El método `listar()` es responsable de recuperar la información de los videojuegos desde la base de datos y transformarla en objetos de las subclases `Accion` o `Terror`. Este proceso incluye una estructura iterativa (bucle `while`) y una estructura de decisión (condicional `if-else`).

2. Código Fuente Analizado

```
public function listar(){
    // NODO 1: Preparación consulta y definición $variantesAccion
    $lista=[];
    $consulta='SELECT * FROM juegos';
    $stmt=$this->db->query($consulta);
    $variantesAccion=['Acción','Accion','acción','accion'];

    // NODO 2: Estructura Iterativa
    while($value=$stmt->fetch(PDO::FETCH_ASSOC)){
        // NODO 3: Estructura Condicional
        if(in_array($value['genero'],$variantesAccion)){
            // NODO 4
            $juego=new Accion(
                $value['nombre'], $value['duracion'], $value['tipoAccion'], $value['t
            );
        } else {
            // NODO 5
            $juego=new Terror(
                $value['nombre'], $value['duracion'], $value['tipoTerror'], $value['t
            );
        }
        // NODO 6
        $lista[]=$juego;
    }
    // NODO 7
    return $lista;
```

```
}
```

3. Análisis del Camino Básico

3.1 Grafo de Flujo de Control

Se identifican los siguientes puntos de ejecución y decisión:

- **1 (Inicio):** Ejecución de la query SQL.
- **2 (Decisión):** ¿Hay más filas que procesar? (Bucle `while`).
- **3 (Decisión):** ¿El género coincide con el array de variantes? (`if`).
- **4 (Proceso):** Instanciación de objeto `Accion`.
- **5 (Proceso):** Instanciación de objeto `Terror`.
- **6 (Proceso):** Inserción en el array y retorno al punto 2.
- **7 (Fin):** Retorno del array final.

3.2 Complejidad Ciclomática

La complejidad ciclomática $V(G)$ representa el número de caminos independientes del fragmento. Se calcula mediante el número de puntos de decisión (P):

$$V(G) = P + 1 = 2 + 1 = 3$$

Esto indica que el plan de pruebas debe contener un mínimo de **3 casos de prueba** para cubrir el camino básico.

4. Plan de Pruebas Unitarias

ID	Camino	Descripción	Entrada (DB)	Resultado Esperado
TC-01	1 → 2 → 7	Tabla vacía: El bucle no llega a entrar.	0 registros.	Array vacío <code>[]</code> .
TC-02	1 → 2 → 3 → 4 → 6 → 2 → 7	Rama Acción: El género es reconocido.	Género = 'acción'	Instancia de <code>Accion</code> .
TC-03	1 → 2 → 3 → 5 → 6 → 2 → 7	Rama Terror: Entra por el bloque Else.	Género = 'Terror'	Instancia de <code>Terror</code> .

5. Conclusiones del plan de pruebas

- Cobertura de Código: Se ha garantizado una cobertura del 100% de las ramas (branch coverage). Los tres casos de prueba diseñados aseguran que tanto el bloque principal del bucle while como las dos ramificaciones del condicional if-else (instanciación de Accion e instanciación de Terror) son ejecutados y verificados.
- Robustez ante Datos Nulos: El caso de prueba TC-01 confirma que el método es capaz de gestionar una base de datos vacía sin lanzar excepciones de tipo "null pointer" o errores de ejecución, retornando una estructura de datos consistente (array vacío).
- Validación de Lógica de Negocio: La prueba de caja blanca permite validar que el array de constantes \$variantesAccion funciona correctamente como filtro, permitiendo que el polimorfismo de la aplicación responda adecuadamente a las distintas entradas de texto (con o sin tildes) provenientes de la base de datos.
- Mantenibilidad: La baja complejidad ciclomática del método ($V(G)=3$) indica que el código es sencillo de mantener y que el riesgo de introducir errores lógicos durante futuras modificaciones es bajo.

Resultado final: El método listar() se considera apto y validado para su paso a producción desde el punto de vista de pruebas unitarias de caja blanca.